GitHub Spec Kit

Spec-Driven Development mit Coding-Agents – ein Mini-Skript

Stand: 26. Oktober 2026 · specify-cli 1.2.3 · Beispiele für VS Code mit GitHub Copilot

1 Worum es geht

Wer mit einem Coding-Agent arbeitet, kennt das Muster: Man beschreibt im Chat, was entstehen soll, der Agent legt los, und nach einigen Runden stimmt das Ergebnis ungefähr. Der Haken ist nicht die Qualität des Codes, sondern dass die **Absicht nirgends festgehalten** ist. Sie lebt im Chatverlauf. Wird der Verlauf lang, verliert der Agent sie; kommt eine Änderung, beginnt das Beschreiben von vorn; steigt ein Mensch dazu, kann er nicht nachlesen, warum etwas so ist.

Spec-Driven Development (SDD) dreht die Reihenfolge um. Zuerst entsteht eine Spezifikation: präzise, gelesen, abgenommen. Daraus wird ein Plan, daraus eine Aufgabenliste, und erst daraus Code. Jede Stufe ist ein Dokument im Repository, das man reviewen kann – wie Code.

> **Der Kerngedanke** Die Spezifikation ist das Primärartefakt, der Code das Abgeleitete. Ändert sich die Anforderung, ändert man die Spezifikation und lässt neu ableiten – nicht umgekehrt.

Spec Kit ist GitHubs quelloffenes Toolkit, das diesen Ablauf in konkrete Vorlagen, Skripte und Chat-Befehle gießt. Es ersetzt keinen Agent, sondern gibt einem vorhandenen Agent einen Prozess.

2 Was Spec Kit konkret ist

Bestandteil	Form	Aufgabe
Specify CLI	Python-Programm `specify`	Projekte anlegen, Integrationen und Erweiterungen verwalten. Läuft im Terminal.
Skills	Markdown-Dateien	Die `/speckit-*`-Befehle. Werden im Chat des Agents ausgeführt, nicht im Terminal.
Templates	Markdown-Vorlagen	Gerüste für Spezifikation, Plan, Aufgabenliste und Checklisten.
Scripts	PowerShell, Bash oder Python	Helfer, die die Skills aufrufen: Feature-Ordner anlegen, Vorbedingungen prüfen.

Das Integrations-Prinzip

Spec Kit ist bewusst **agent-agnostisch**. Eine *Integration* entscheidet allein, in welches Verzeichnis und in welchem Format die Skill-Dateien geschrieben werden – der Prozess und die Befehlsnamen bleiben identisch. Mehr als 40 Integrationen sind eingebaut, unter anderem `copilot`, `claude`, `cursor-agent`, `gemini`, `codex` sowie none für ein agent-neutrales Projekt.

Anatomie eines Skills

Ein Skill ist nichts Geheimnisvolles, sondern eine Textdatei. Der Platzhalter `$ARGUMENTS` nimmt auf, was hinter dem Slash-Befehl getippt wurde:

Auszug aus `.github/skills/speckit-specify/SKILL.md`

```
---
name: "speckit-specify"
description: "Create or update the feature specification from a
              natural language feature description."
compatibility: "Requires spec-kit project structure with .specify/"
---

## User Input

```text
$ARGUMENTS
```

You **MUST** consider the user input before proceeding ...
```

Genau deshalb funktioniert dasselbe Spec Kit mit so vielen Agents: Es liefert Prompts, keine Programmlogik.

3 Installation und Projekt-Setup

Benötigt werden Python 3.11 oder neuer, der Paketmanager *uv*, sowie ein unterstützter Coding-Agent. Linux, macOS und Windows sind gleichwertig.

Terminal

```
uv tool install specify-cli
specify init mein-projekt --integration copilot --script ps
cd mein-projekt
```

Flag	Bedeutung
`--integration <key>`	Für welchen Agent die Skills installiert werden. Ohne Angabe fragt `init` interaktiv.
`--script sh|ps|py`	Sprache der Helfer-Skripte. Auf Windows ist `ps` die richtige Wahl.
`--non-interactive`	Überspringt alle Rückfragen und nimmt Vorgaben. Für CI und Automatisierung.
`--integration-options`	Integrationsspezifische Schalter, bei Copilot etwa `"--commands"` für das alternative Prompt-Layout.

Für ein **bestehendes Repository** gilt derselbe Befehl mit `.` als Ziel. Arbeitet ein **Team mit verschiedenen Agents**, legt man das Projekt mit `--integration none` neutral an; jede Person schaltet lokal per `specify integration switch <key>` um und vor dem Commit wieder zurück.

4 Projektstruktur

Verzeichnisbaum nach dem ersten Feature

```
mein-projekt/
+- .github/
|  \- skills/                    # die Slash-Befehle (hier: Copilot)
|     +- speckit-constitution/SKILL.md
|     +- speckit-specify/SKILL.md
|     +- ... (insgesamt 10 Skills)
+- .specify/
|  +- memory/constitution.md     # Grundregeln des Projekts
|  +- templates/                 # Vorlagen für spec, plan, tasks
|  +- scripts/powershell/        # Helfer-Skripte
|  +- workflows/                 # optionale Automatisierung
|  +- integration.json           # welcher Agent ist aktiv
|  \- feature.json               # welches Feature ist aktiv
\- specs/
   \- 001-mein-feature/          # je Feature ein Ordner
      +- spec.md
      +- plan.md
      \- tasks.md
```

Nicht von Hand editieren Die generierten Skill-Dateien werden bei `specify integration upgrade` überschrieben. Anpassungen gehören in *Presets* oder *project-local overrides* (Abschnitt 8).

5 Der SDD-Workflow

Alle folgenden Befehle laufen im **Chat des Agents**. Die Regel lautet: **ein Skill pro Aufruf, dann das Ergebnis lesen.** Wer alle neun hintereinander abfeuert, ohne die Artefakte zu prüfen, hat nur eine umständlichere Art gefunden, dem Agent „bau mir mal X“ zu sagen.

#	Skill	Zweck	Erzeugt
1	`/speckit-constitution`	Grundregeln des Projekts festlegen. Einmal pro Projekt. Alle späteren Schritte werden dagegen geprüft.	constitution.md
2	`/speckit-specify`	Das **Was** und **Warum** beschreiben – ohne jede Technik.	spec.md
3	`/speckit-clarify`	Stellt Rückfragen zu Unterspezifiziertem und arbeitet die Antworten ein. *Optional.*	aktualisierte spec.md
4	`/speckit-plan`	Das **Wie**: Technologie, Architektur, Dateiaufteilung.	plan.md, research.md, data-model.md
5	`/speckit-checklist`	Qualitäts-Checkliste für die Spezifikation. *Optional.*	checklists/*.md
6	`/speckit-tasks`	Zerlegt den Plan in abhängigkeitssortierte Schritte.	tasks.md
7	`/speckit-analyze`	Prüft spec, plan und tasks gegeneinander auf Widersprüche. Nur lesend. *Optional.*	Bericht im Chat
8	`/speckit-implement`	Arbeitet tasks.md ab und schreibt den Code.	Quellcode, Tests
9	`/speckit-converge`	Prüft den Code gegen spec, plan und tasks. Findet er Lücken, hängt er neue Aufgaben an.	ergänzte tasks.md

Kurzpfad und Vollpfad

Für kleine Features genügen fünf Schritte: `specify`, `plan`, `tasks`, `implement`, `converge`. Die drei optionalen Schritte `clarify`, `checklist` und `analyze` sind Qualitätsgates für alles, was in Produktion geht.

Die Konvergenz-Schleife

Schritt 8 und 9 bilden einen Zyklus: implementieren, prüfen, ergänzen, wieder implementieren. Die Abbruchbedingung ist die Meldung **Converged** – erst dann gilt das Feature als fertig. Das ersetzt das Bauchgefühl „das sieht gut aus“ durch ein prüfbares Kriterium.

6 Die vier Artefakte – und was nicht hineingehört

Der häufigste Anwendungsfehler ist das Vermischen der Ebenen: Technik in der Spezifikation, Anforderungen im Plan. Diese Tabelle ist der praktische Kern des Verfahrens.

Artefakt	Beantwortet	Gehört nicht hinein
`constitution.md`	Welche Regeln gelten im ganzen Projekt? Sprachversion, Teststrategie, Dokumentationspflicht, Abhängigkeitspolitik.	Alles, was nur ein einzelnes Feature betrifft.
`spec.md`	Wer braucht was, und warum? Nutzergeschichten, funktionale Anforderungen, Abnahmekriterien, offene Punkte.	Frameworks, Bibliotheken, Datenbanken, Dateinamen, Architektur.
`plan.md`	Wie wird es gebaut? Technologie, Module, Schnittstellen, Datenmodell, Teststrategie für dieses Feature.	Neue Anforderungen. Taucht hier eine auf, gehört sie in die Spezifikation.
`tasks.md`	In welcher Reihenfolge? Nummerierte, abhängigkeitssortierte Arbeitsschritte.	Begründungen und Entscheidungen – die stehen in spec und plan.

Prüffrage beim Schreiben Lässt sich ein Satz auch sinnvoll sagen, wenn man die Technologie austauscht? Dann gehört er in die Spezifikation. Sonst in den Plan.

7 Feature-Verwaltung

Jedes Feature erhält einen nummerierten Ordner unter `specs/`, etwa `specs/001-mein-feature/`. Welches Feature gerade aktiv ist, steht in `.specify/feature.json` und lässt sich über die Umgebungsvariable `SPECIFY_FEATURE_DIRECTORY` überschreiben.

Häufiges Missverständnis Frühere Versionen banden das aktive Feature an den Git-Branch. Heute nicht mehr: *Git ist nicht erforderlich*, und `git checkout` wechselt das aktive Feature **nicht**. Nummerierte Feature-Branches gibt es weiterhin, aber nur als Opt-in über `specify extension add git`.

8 Die beiden anderen Prozesse

Seit Version 1.0 ist Spec Kit mehr als SDD. Zwei weitere Prozesse lassen sich als Erweiterung nachinstallieren; beide sind **eigenständige Einstiegspunkte** und setzen keinen SDD-Durchlauf voraus.

Prozess	Installation	Skills	Ergebnis
Bug-Fixing	`specify extension add bug`	`/speckit-bug-assess` `/speckit-bug-fix` `/speckit-bug-test`	Bericht in `.specify/bugs/<slug>/` mit Verdikt **verified**, **partial** oder **failed**. Nur *verified* bestätigt die Behebung.
Ideen-Bewertung	`specify extension add assess`	`/speckit-assess-intake` `/speckit-assess-research` `/speckit-assess-define` `/speckit-assess-shape` `/speckit-assess-decide`	Artefakte in `.specify/assessments/<slug>/` mit Entscheidung **go**, **needs-clarification** oder **kill**. Ein begründetes Nein ist ein nützliches Ergebnis.

Anpassungsmechanismen

Mechanismus	Wofür
Extension	Einen Befehl, ein Qualitätsgate oder einen ganzen Prozess hinzufügen.
Preset	Templates, Befehle oder Terminologie anpassen – etwa um regulatorische Nachweispflichten zu erzwingen.
Project-local override	Eine einzelne Vorlage für genau ein Projekt ersetzen.
Workflow	Prompts, Befehle, Gates und Shell-Schritte automatisieren. *Overlays* passen einen Workflow an, ohne ihn zu forken.
Bundle	Ein fertiges Setup für eine Rolle oder ein Team paketieren.
Catalog	Komponenten entdecken – oder einen firmeneigenen, geprüften Katalog betreiben.

Gesucht wird mit `specify extension search`, `specify preset search`, `specify workflow search` und `specify bundle search`.

> **Sicherheitshinweis** Ein Katalog-Eintrag ist keine Prüfung und keine Empfehlung. Quellcode von Komponenten aus Gemeinschaftskatalogen vor der Installation lesen. Und: Manche Agents legen Zugangsdaten im Agent-Ordner ab – vor dem Pushen prüfen, was aus `.github/` wirklich ins Repository gehört.

9 Praxisregeln

- **Ein Skill pro Aufruf.** Danach das Ergebnis lesen, bevor der nächste folgt.
- **In der Spezifikation keine Technik.** Kein Framework, keine Bibliothek, keine Dateinamen. Die gehören in den Plan.
- **Explizit sein.** Was nicht dasteht, wird geraten. Lieber einen Satz zu viel über Randfälle als eine Annahme zu viel im Code.
- **Vor dem Planen klären.** `/speckit-clarify` deckt auf, wie viele Entscheidungen man implizit offen gelassen hat.
- **Widersprüche an der Quelle beheben.** Meldet `/speckit-analyze` etwas, korrigiert man spec oder plan und lässt neu laufen – nicht beim Implementieren übergehen.
- **Bis Converged iterieren.** Die Meldung ist das Abnahmekriterium, nicht der Eindruck, dass es läuft.
- **Artefakte versionieren.** spec, plan und tasks gehören ins Repository und in den Review – sie sind die eigentliche Dokumentation.

10 Stolpersteine

Symptom	Ursache	Abhilfe
Die `/speckit-*`-Befehle erscheinen nicht im Chat.	Meist der falsche Workspace-Root, der falsche Chat-Modus oder ein nicht neugeladenes Fenster.	Projektordner selbst öffnen (nicht den Elternordner); Chat in den **Agent**-Modus schalten; *Developer: Reload Window*; dann `specify integration status` und notfalls `specify integration upgrade <key>`.
Befehle heißen in Blogposts `/speckit.specify` mit Punkt.	Veraltete Quellen. Auch `uvx --from git+...` und `--ai copilot` sind überholt.	Aktuell: `uv tool install specify-cli`, `--integration <key>`, und Skills mit **Bindestrich**.
Der Agent arbeitet am falschen Feature.	Das aktive Feature hängt an `feature.json`, nicht am Branch.	`.specify/feature.json` anpassen oder `SPECIFY_FEATURE_DIRECTORY` setzen.
Eigene Änderungen an Skills sind verschwunden.	Generierte Dateien werden beim Upgrade ersetzt.	Presets oder project-local overrides verwenden.
Das Ergebnis ist so beliebig wie ohne Spec Kit.	Die Artefakte wurden nicht gelesen.	Spec Kit ist ein Review-Verfahren, keine Pipeline. Der Nutzen entsteht beim Prüfen, nicht beim Generieren.

11 Befehlsreferenz

Im Terminal

Befehl	Wirkung
`uv tool install specify-cli`	CLI installieren.
`specify init <name> --integration <key>`	Neues Projekt anlegen.
`specify integration list`	Verfügbare Integrationen und Installationsstatus.
`specify integration status`	Status im aktuellen Projekt, ohne etwas zu ändern.
`specify integration install <key>`	Weiteren Agent hinzufügen.
`specify integration use <key>`	Standard-Agent setzen, ohne andere zu entfernen.
`specify integration switch <key>`	Integration ersetzen.
`specify integration upgrade <key>`	Integration neu installieren, diff-bewusst.
`specify extension add <name>`	Erweiterung installieren (z. B. bug, assess, git).
`specify extension list`	Installierte Erweiterungen.
`specify extension search <begriff>`	Katalog durchsuchen.

Im Chat des Agents

Skill	Prozess	Rolle
`/speckit-constitution`	SDD	Grundregeln, einmal pro Projekt
`/speckit-specify`	SDD	Spezifikation – das Was
`/speckit-clarify`	SDD	Rückfragen, optional
`/speckit-plan`	SDD	Plan – das Wie
`/speckit-checklist`	SDD	Qualitätscheck der Spezifikation, optional
`/speckit-tasks`	SDD	Aufgabenliste
`/speckit-analyze`	SDD	Konsistenzprüfung, optional
`/speckit-implement`	SDD	Implementierung
`/speckit-converge`	SDD	Abgleich bis *Converged*
`/speckit-taskstoissues`	SDD	GitHub Issues aus tasks.md erzeugen
`/speckit-bug-*`	Bug-Fixing	assess, fix, test
`/speckit-assess-*`	Bewertung	intake, research, define, shape, decide

12 Quellen

- Projekt und Quellcode: `github.com/github/spec-kit`
- Dokumentation: `github.github.io/spec-kit`
- SDD-Schnelleinstieg: `github.github.io/spec-kit/quickstart.html`
- Befehlsreferenz: `github.github.io/spec-kit/reference/agentic-sdd.html`
- Vollständige Methodik: `spec-driven.md` im Repository

Spec Kit steht unter MIT-Lizenz. Dieses Skript wurde am 26. Oktober 2026 gegen specify-cli 1.2.3 geprüft; Versionsangaben und Befehlsnamen ändern sich mit neuen Releases.